

Reflections for BreakAPlate program

How did you approach understanding the challenge?

By analysing the challenge, and through time I was coding I started to understand the purposes of the challenge better. I made sure to understand the purpose of each method and action listeners in the java code and how it connects to the GUI design.

How did you decide on the tools, data structures, or algorithms to use?

I was looking for the explanation of the challenge and then I was focused on using the GUI Java libraries. I used the components and listeners provided, like JFrame, JButton, JLabel and ActionListener. I didn't need complex data structures, but I used event listeners to handle responses from the JButton and an algorithm Math.random to create a random number.

```
JFrame frame;
```

```
JLabel plates = new JLabel(plate);  
plates.setBounds(10, 11, 414, 104);  
frame.getContentPane().add(plates);  
  
// Label to display the prize image or placeholder in the bottom area.  
JLabel prizes = new JLabel(" ");  
prizes.setHorizontalAlignment(SwingConstants.CENTER);  
prizes.setBounds(50, 151, 325, 110);  
frame.getContentPane().add(prizes);  
  
// Button the user clicks to play or play again.  
JButton play = new JButton("Play");
```

```
play.addActionListener(new ActionListener() {
```

How did you test your solution along the way to make sure it was working?

I ran the program during my coding to check for the errors and fix them if needed. I also used a preview of my design to check if it was right.

What part of the problem was the most difficult for you?

The most difficult part was understanding how the event listeners worked and ensuring the correct image was shown when a Button was clicked.

Was there anything you learned that you think will help you with future challenges?

I learned the importance of event driven programming. I also became more confident while using external libraries in Java.

```
play.addActionListener(new ActionListener() {  
    public void actionPerformed(ActionEvent e)  
    {
```

Reflections for TicTacToe program

How did you approach understanding the challenge?

By analysing the challenge, and through time I was coding I started to understand the purposes of the challenge better. I made sure to understand the purpose of each method and action listeners in the java code and how it connects to the GUI design.

How did you decide on the tools, data structures, or algorithms to use?

I was looking for the explanation of the challenge and then I was focused on using the GUI Java libraries. I used the components and listeners provided, like JFrame, JButton, JLabel, and ActionListener. I didn't need complex data structures or algorithms, but I used event listeners to handle responses from the JButton.

```
// Main application window.  
private JFrame frame;  
  
// Buttons for starting/resetting the game and for each of the 9 grid squares.  
private JButton startbtn, btn1, btn2, btn3, btn4, btn5, btn6, btn7, btn8, btn9;  
// Label to display the winner or whose turn it is.  
JLabel winner;
```

How did you test your solution along the way to make sure it was working?

I ran the program during my coding to check for the errors and fix them if needed. I also used a preview of my design to check if it was right.

What part of the problem was the most difficult for you?

The most difficult part was to create a method that checks if the user won after the button is clicked, as well as adding a listener for all 9 buttons.

```
// Method to check all possible winning combinations and return a message if X or O wins.  
public String checkWinner()  
{  
  
    // Rows, columns, and diagonals for player X.  
    if(b1.equals("X") && b2.equals("X") && b3.equals("X")) return "The Winner Is X!";  
    if(b4.equals("X") && b5.equals("X") && b6.equals("X")) return "The Winner Is X!";  
    if(b7.equals("X") && b8.equals("X") && b9.equals("X")) return "The Winner Is X!";  
    if(b1.equals("X") && b5.equals("X") && b9.equals("X")) return "The Winner Is X!";  
    if(b7.equals("X") && b5.equals("X") && b3.equals("X")) return "The Winner Is X!";  
    if(b1.equals("X") && b4.equals("X") && b7.equals("X")) return "The Winner Is X!";  
    if(b2.equals("X") && b5.equals("X") && b8.equals("X")) return "The Winner Is X!";  
    if(b3.equals("X") && b6.equals("X") && b9.equals("X")) return "The Winner Is X!";  
  
    // Rows, columns, and diagonals for player O.  
    if(b1.equals("O") && b2.equals("O") && b3.equals("O")) return "The Winner Is O!";  
    if(b4.equals("O") && b5.equals("O") && b6.equals("O")) return "The Winner Is O!";  
    if(b7.equals("O") && b8.equals("O") && b9.equals("O")) return "The Winner Is O!";  
    if(b1.equals("O") && b5.equals("O") && b9.equals("O")) return "The Winner Is O!";  
    if(b7.equals("O") && b5.equals("O") && b3.equals("O")) return "The Winner Is O!";  
    if(b1.equals("O") && b4.equals("O") && b7.equals("O")) return "The Winner Is O!";  
    if(b2.equals("O") && b5.equals("O") && b8.equals("O")) return "The Winner Is O!";  
    if(b3.equals("O") && b6.equals("O") && b9.equals("O")) return "The Winner Is O!";  
  
    // No winner yet.  
    return " ";  
}
```

Was there anything you learned that you think will help you with future challenges?

I learned how to design the program and how to properly use event listeners in a complex program.

```
btn1.addActionListener(new ActionListener() {  
    public void actionPerformed(ActionEvent e)  
    {  
        // If it's X's turn and this cell is not already 0, place X.  
        if(turn == 0 && !btn1.getText().equals("0"))  
        {  
            btn1.setText("X");  
            winner.setText("Second player turn(0)");  
            b1 = "X";  
            // After the move, check if there is a winner.  
            if(!checkWinner().equals(" "))  
            {  
                disableButtons();  
                winner.setText(checkWinner());  
            }  
            // Switch turn to 0.  
            turn += 1;  
        }  
  
        // If it's 0's turn and this cell is not already X, place 0.  
        else if(turn == 1 && !btn1.getText().equals("X"))  
        {  
            btn1.setText("0");  
            winner.setText("First player turn(X)");  
            b1 = "0";  
            // After the move, check if there is a winner.  
            if(!checkWinner().equals(" "))  
            {  
                disableButtons();  
                winner.setText(checkWinner());  
            }  
            // Switch turn back to X.  
            turn -= 1;  
        }  
    }  
});
```

Reflections for LocalBank program

How did you approach understanding the challenge?

By analysing the challenge, and through time I was coding I started to understand the purposes of the challenge better. I made sure to understand the purpose of each method and action listeners in the java code and how it connects to the GUI design.

How did you decide on the tools, data structures, or algorithms to use?

I was looking for the explanation of the challenge and then I was focused on using the GUI Java libraries. I used the components and listeners provided, like JFrame, JButton, JLabel, JTextField, JTextArea, JComboBox and ActionListeners. I used algorithms such as parseDouble() to change String value into a double value, isEmpty() to check if the TextFields are empty or not and trim() to get rid of spacing before and after users input in the TextField. Also, I used event listeners to handle responses from the users input.

```
private JFrame frame;  
private JTextField account;  
private JTextField amount;  
private JTextField firstN;  
private JTextField lastN;  
private JTextField balance;  
JComboBox action;  
JTextArea accountinfo, error;  
JLabel actionlbl, accountlbl, amountlbl, firstNlbl, lastNlbl, balancelbl, infolbl;  
JButton process;
```

```
String fNameInput = firstN.getText().trim();  
String lNameInput = lastN.getText().trim();  
String balInput = balance.getText().trim();  
String accountInput = account.getText().trim();  
String amountInput = amount.getText().trim();
```

```
if((fNameInput.isEmpty() || lNameInput.isEmpty() || balInput.isEmpty())
```

```
double balValue = Double.parseDouble(balInput);
```

How did you test your solution along the way to make sure it was working?

I ran the program during my coding to check for the errors and fix them if needed. I also used a preview of my design to check if it was right.

What part of the problem was the most difficult for you?

The most difficult part was using action listeners to get what the user wants to do, and running the needed code for the action chosen. Moreover, I had a problem with finding a way to take the user's response from the textfields. So I decided to use a String variable and .trim() algorithm to store the user's response.

```
if((fNameInput.isEmpty() || lNameInput.isEmpty() || balInput.isEmpty()) && action.getSelectedItem().equals("Add Account"))
```

```
String fNameInput = firstN.getText().trim();  
String lNameInput = lastN.getText().trim();  
String balInput = balance.getText().trim();  
String accountInput = account.getText().trim();  
String amountInput = amount.getText().trim();
```

Was there anything you learned that you think will help you with future challenges?

I learned the way to store user's input from the textfield, I also learned how to run the code needed for the chosen action.

```
// If the user selected "Add Account", create a new account
```

```
if(action.getSelectedItem().equals("Add Account"))  
{
```

```
String fNameInput = firstN.getText().trim();  
String lNameInput = lastN.getText().trim();  
String balInput = balance.getText().trim();  
String accountInput = account.getText().trim();  
String amountInput = amount.getText().trim();
```